

StringBuilder and StringBuffer in Java

In Java, both `StringBuilder` and `StringBuffer` are used to create mutable sequences of characters. Unlike the `String` class, which is **immutable**, the `StringBuilder` and `StringBuffer` classes allow modifications to the existing string without creating a new object each time. These classes are particularly useful when performing numerous string manipulations.

Key Differences Between StringBuilder and StringBuffer

- **Thread Safety:**
 - **StringBuffer:** Synchronized, hence thread-safe. It is slower compared to `StringBuilder` due to the synchronization overhead.
 - **StringBuilder:** Not synchronized, hence not thread-safe. It is faster than `StringBuffer` because it does not have the synchronization overhead.
 - **Performance:**
 - **StringBuilder** is generally preferred when thread safety is not a concern because of its better performance.
 - **StringBuffer** is used when thread safety is required, though it tends to be slower due to synchronization.
-

StringBuilder Class

The `StringBuilder` class is used to create a mutable sequence of characters. It is part of the `java.lang` package.

Commonly Used Methods in StringBuilder

1. `append(String str)`

- Appends the specified string to the current string.
- **Syntax:**

```
StringBuilder append(String str);
```

- **Example:**

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");
System.out.println(sb); // Output: "Hello World"
```

2. insert(int offset, String str)

- Inserts the specified string at the specified position.
- **Syntax:**

```
StringBuilder insert(int offset, String str);
```

- **Example:**

```
StringBuilder sb = new StringBuilder("Hello World");
sb.insert(5, ",");
System.out.println(sb); // Output: "Hello, World"
```

3. delete(int start, int end)

- Deletes a substring from the current string. The substring is defined by the start and end indices.
- **Syntax:**

```
StringBuilder delete(int start, int end);
```

- **Example:**

```
StringBuilder sb = new StringBuilder("Hello World");
sb.delete(5, 11);
System.out.println(sb); // Output: "Hello"
```

4. reverse()

- Reverses the characters in the `StringBuilder`.
- **Syntax:**

```
StringBuilder reverse();
```

- **Example:**

```
StringBuilder sb = new StringBuilder("Hello");
sb.reverse();
System.out.println(sb); // Output: "olleH"
```

5. toString()

- Converts the `StringBuilder` object to a `String`.
- **Syntax:**

```
String toString();
```

- **Example:**

```
StringBuilder sb = new StringBuilder("Hello");  
String str = sb.toString();  
System.out.println(str); // Output: "Hello"
```

6. capacity()

- Returns the current capacity of the `StringBuilder`.
- **Syntax:**

```
int capacity();
```

- **Example:**

```
StringBuilder sb = new StringBuilder("Hello");  
System.out.println(sb.capacity()); // Output: 16 (default capacity)
```

7. setLength(int newLength)

- Sets the length of the `StringBuilder` to the specified length. If the new length is shorter than the current length, characters are deleted; if longer, the `StringBuilder` is padded with null characters.
- **Syntax:**

```
void setLength(int newLength);
```

- **Example:**

```
StringBuilder sb = new StringBuilder("Hello World");  
sb.setLength(5);  
System.out.println(sb); // Output: "Hello"
```

8. replace(int start, int end, String str)

- Replaces the characters in the specified range with the given string.
- **Syntax:**

```
StringBuilder replace(int start, int end, String str);
```

- **Example:**

```
StringBuilder sb = new StringBuilder("Hello World");  
sb.replace(6, 11, "Java");  
System.out.println(sb); // Output: "Hello Java"
```

StringBuffer Class

The `StringBuffer` class is similar to `StringBuilder` but is **thread-safe**. It is part of the `java.lang` package, and its methods are synchronized, so multiple threads can safely use it.

Commonly Used Methods in StringBuffer

1. `append(String str)`

- Appends the specified string to the current string.
- **Syntax:**

```
StringBuffer append(String str);
```

- **Example:**

```
StringBuffer sb = new StringBuffer("Hello");  
sb.append(" World");  
System.out.println(sb); // Output: "Hello World"
```

2. `insert(int offset, String str)`

- Inserts the specified string at the specified position.
- **Syntax:**

```
StringBuffer insert(int offset, String str);
```

- **Example:**

```
StringBuffer sb = new StringBuffer("Hello World");
sb.insert(5, ",");
System.out.println(sb); // Output: "Hello, World"
```

3. delete(int start, int end)

- Deletes a substring from the current string. The substring is defined by the start and end indices.
- **Syntax:**

```
StringBuffer delete(int start, int end);
```

- **Example:**

```
StringBuffer sb = new StringBuffer("Hello World");
sb.delete(5, 11);
System.out.println(sb); // Output: "Hello"
```

4. reverse()

- Reverses the characters in the `StringBuffer`.
- **Syntax:**

```
StringBuffer reverse();
```

- **Example:**

```
StringBuffer sb = new StringBuffer("Hello");
sb.reverse();
System.out.println(sb); // Output: "olleH"
```

5. toString()

- Converts the `StringBuffer` object to a `String`.
- **Syntax:**

```
String toString();
```

- **Example:**

```
StringBuffer sb = new StringBuffer("Hello");
String str = sb.toString();
System.out.println(str); // Output: "Hello"
```

6. capacity()

- Returns the current capacity of the `StringBuffer`.
- **Syntax:**

```
int capacity();
```

- **Example:**

```
StringBuffer sb = new StringBuffer("Hello");  
System.out.println(sb.capacity()); // Output: 16 (default capacity)
```

7. setLength(int newLength)

- Sets the length of the `StringBuffer` to the specified length. If the new length is shorter than the current length, characters are deleted; if longer, the `StringBuffer` is padded with null characters.
- **Syntax:**

```
void setLength(int newLength);
```

- **Example:**

```
StringBuffer sb = new StringBuffer("Hello World");  
sb.setLength(5);  
System.out.println(sb); // Output: "Hello"
```

8. replace(int start, int end, String str)

- Replaces the characters in the specified range with the given string.
- **Syntax:**

```
StringBuffer replace(int start, int end, String str);
```

- **Example:**

```
StringBuffer sb = new StringBuffer("Hello World");  
sb.replace(6, 11, "Java");  
System.out.println(sb); // Output: "Hello Java"
```

Example: StringBuilder and StringBuffer in Action

```
public class StringBuilderBufferExample {
    public static void main(String[] args) {
        // Using StringBuilder
        StringBuilder sb = new StringBuilder("Hello");
        sb.append(" World");
        sb.insert(5, ",");
        System.out.println(sb); // Output: "Hello, World"

        sb.reverse();
        System.out.println(sb); // Output: "dlroW ,olleH"

        sb.delete(5, 12);
        System.out.println(sb); // Output: "dlroW"

        // Using StringBuffer
        StringBuffer sbf = new StringBuffer("Hello");
        sbf.append(" World");
        sbf.insert(5, ",");
        System.out.println(sbf); // Output: "Hello, World"

        sbf.reverse();
        System.out.println(sbf); // Output: "dlroW ,olleH"

        sbf.delete(5, 12);
        System.out.println(sbf); // Output: "dlroW"
    }
}
```

Output:

```
Hello, World
dlroW ,olleH
dlroW
Hello, World
dlroW ,olleH
dlroW
```
